

JAVA TRANSACTION ROLLBACK WITHOUT @Transactional

Handwritten-style senior Java / Spring interview study notes

Interview Question

Question: How do you implement rollback in a Java/Spring application when `@Transactional` is not available or cannot be used?

This document explains the answer at a **senior Java backend developer level**, including:

- Local database rollback using JDBC
- Programmatic transaction management using Spring
- `TransactionTemplate`
- `PlatformTransactionManager`
- Multiple database scenarios
- XA/JTA and Two-Phase Commit (2PC)
- Saga pattern and compensating transactions
- Idempotency, retries, and persistent workflow state
- Recommended interview answers and follow-up questions

1. Core Concept

A transaction groups multiple operations into one logical unit of work.

The expected behavior is generally:

```
BEGIN TRANSACTION
  Operation 1
  Operation 2
  Operation 3

If all operations succeed:
  COMMIT

If any operation fails:
  ROLLBACK
```

In Spring, `@Transactional` normally provides this behavior declaratively.

Example:

```
@Transactional
public void createOrder() {
    orderRepository.save(order);
    paymentRepository.save(payment);
}
```

If `@Transactional` is unavailable, the same transaction behavior can still be implemented programmatically.

The solution depends on the architecture.

Scenario 1: One database

-> JDBC / TransactionTemplate / PlatformTransactionManager

Scenario 2: Multiple databases participating in one atomic transaction

-> XA / JTA / Two-Phase Commit

Scenario 3: Multiple microservices with separate databases

-> Saga + Compensation + Retry + Idempotency

2. Option 1 - Plain JDBC Transaction Management

At the JDBC level, transaction management is performed through `java.sql.Connection`.

Example

```
public void createOrder(DataSource dataSource) throws SQLException {  
    Connection connection = null;  
  
    try {  
        connection = dataSource.getConnection();  
  
        // Disable automatic commit.  
        connection.setAutoCommit(false);  
  
        insertOrder(connection);  
        insertOrderItems(connection);  
        updateInventory(connection);  
  
        // Commit only when every operation succeeds.  
        connection.commit();  
    } catch (Exception ex) {  
        if (connection != null) {  
            connection.rollback();  
        }  
  
        throw new RuntimeException("Order creation failed", ex);  
    } finally {  
        if (connection != null) {  
            connection.close();  
        }  
    }  
}
```

Flow



Important Point

All operations must use the **same JDBC connection** if they are expected to be part of the same local database transaction.

If each repository obtains a different independent connection, the operations may not participate in the same transaction.

When to use

Plain JDBC transaction management is useful when:

- The application is not using Spring transaction management.
- You require low-level control.
- You are using JDBC directly.

In a typical Spring application, programmatic Spring transaction management is usually cleaner.

3. Option 2 - Spring `TransactionTemplate`

If `@Transactional` cannot be used, one of the cleanest Spring alternatives is `TransactionTemplate`.

`TransactionTemplate` provides programmatic transaction management while still allowing Spring to manage transaction resources.

Example

```
@Service
public class OrderService {

    private final TransactionTemplate transactionTemplate;
    private final OrderRepository orderRepository;
    private final PaymentRepository paymentRepository;

    public OrderService(
        PlatformTransactionManager transactionManager,
        OrderRepository orderRepository,
        PaymentRepository paymentRepository) {

        this.transactionTemplate = new TransactionTemplate(transactionManager);
        this.orderRepository = orderRepository;
        this.paymentRepository = paymentRepository;
    }

    public void createOrder(Order order, Payment payment) {

        transactionTemplate.executeWithoutResult(status -> {

            try {
                orderRepository.save(order);
                paymentRepository.save(payment);
            } catch (Exception ex) {

                status.setRollbackOnly();
                throw ex;
            }
        });
    }
}
```

Behavior

```
TransactionTemplate starts transaction
|
v
orderRepository.save()
|
v
paymentRepository.save()
|
+---+---+
|           |
Success     Exception
|           |
COMMIT      ROLLBACK
```

Why it is useful

Compared with manual JDBC transaction handling, Spring takes care of:

- Transaction resource binding
- Connection lifecycle
- Commit
- Rollback
- Integration with configured transaction managers

- Transaction propagation behavior

Senior-Level Answer

If `@Transactional` cannot be used, I normally prefer `TransactionTemplate` for programmatic transaction boundaries because it gives explicit control while still using Spring's transaction infrastructure.

4. Option 3 - PlatformTransactionManager

Another option is to interact directly with Spring's `PlatformTransactionManager`.

This provides even more explicit control than `TransactionTemplate`.

Example

```
@Service
public class OrderService {

    private final PlatformTransactionManager transactionManager;
    private final OrderRepository orderRepository;
    private final PaymentRepository paymentRepository;

    public OrderService(
        PlatformTransactionManager transactionManager,
        OrderRepository orderRepository,
        PaymentRepository paymentRepository) {

        this.transactionManager = transactionManager;
        this.orderRepository = orderRepository;
        this.paymentRepository = paymentRepository;
    }

    public void createOrder(Order order, Payment payment) {

        DefaultTransactionDefinition definition =
            new DefaultTransactionDefinition();

        TransactionStatus status =
            transactionManager.getTransaction(definition);

        try {
            orderRepository.save(order);
            paymentRepository.save(payment);

            transactionManager.commit(status);
        } catch (Exception ex) {

            transactionManager.rollback(status);
            throw ex;
        }
    }
}
```

Comparison

```
@Transactional
    -> Declarative

TransactionTemplate
    -> Programmatic, simpler

PlatformTransactionManager
    -> Programmatic, lower-level control
```

For most programmatic transaction requirements, `TransactionTemplate` is easier to maintain.

5. What Happens When an Exception Occurs?

Consider:

```
Transaction starts

Insert Order      -> SUCCESS
Insert Payment    -> SUCCESS
Update Inventory  -> FAILURE
```

If all operations are participating in the same local transaction:

```
Insert Order      -> rolled back
Insert Payment    -> rolled back
Update Inventory  -> rolled back
```

The database returns to its state before the transaction started.

6. Important Interview Follow-Up: DB1 Succeeds but DB2 Fails

The interviewer may ask:

What happens if DB1 succeeds but DB2 fails?

The answer depends on whether DB1 and DB2 participate in the same distributed transaction.

There are two major approaches.

```
Approach A: XA/JTA / Two-Phase Commit

Approach B: Saga / Compensating Transactions
```

7. Scenario A - XA/JTA with Two-Phase Commit

XA/JTA is used when multiple transactional resources must behave like one atomic transaction.

Example:

```
Application
|
+---- DB1: Order Database
|
+---- DB2: Accounting Database
```

Business operation:

1. Insert order into DB1
2. Insert accounting entry into DB2

Requirement:

```
Either:

DB1 COMMIT
DB2 COMMIT

OR

DB1 ROLLBACK
DB2 ROLLBACK
```

We do not want this situation:

```
DB1 COMMIT -> SUCCESS
DB2 COMMIT -> FAILURE

Result: inconsistent state
```

XA solves this using a distributed transaction manager and **Two-Phase Commit (2PC)**.

8. What Is JTA?

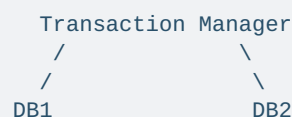
JTA stands for:

Java Transaction API

It defines APIs for coordinating distributed transactions across multiple transactional resources.

The transaction manager coordinates the participating resources.

Conceptually:



Each database must support participation in the distributed transaction, generally through an XA-capable driver/resource.

9. Two-Phase Commit - Phase 1: Prepare

The transaction manager first asks every participating resource whether it is ready to commit.

```
Transaction Manager
|
+----- DB1: PREPARE?
|         |
|         +----- YES
|
+----- DB2: PREPARE?
|         |
|         +----- YES
```

At this point the participants have not completed the final commit.

They have indicated that they are prepared to commit if instructed.

10. Two-Phase Commit - Phase 2: Commit

If every resource successfully prepares:

```
Transaction Manager
|
+----- DB1: COMMIT
|
+----- DB2: COMMIT
```

Final result:

```
DB1 -> COMMITTED
DB2 -> COMMITTED
```

11. What If DB2 Cannot Prepare?

Example:

```
DB1 -> PREPARE -> YES
DB2 -> PREPARE -> NO
```

The transaction manager instructs participants to roll back.

```
Transaction Manager
|
+----- DB1 -> ROLLBACK
|
+----- DB2 -> ROLLBACK
```

Result:

```
DB1 -> ROLLED BACK
DB2 -> ROLLED BACK
```

This provides atomicity across participating resources.

12. Why XA/JTA Is Not Always Preferred

XA/2PC gives strong transactional guarantees, but it comes with trade-offs.

Potential disadvantages include:

- Greater infrastructure complexity
- Distributed transaction coordinator dependency
- Higher coupling between participants
- Longer-held database resources/locks
- Reduced availability during coordinator/resource failures
- More difficult operational troubleshooting
- Scalability limitations compared with independent local transactions

Because of these trade-offs, XA is less common in loosely coupled microservice architectures.

It can still be appropriate when strict distributed atomicity is required and all involved resources support the protocol reliably.

13. Scenario B - Saga Pattern

The Saga pattern is commonly used in microservice architectures where each service owns its own database.

Example:

```
Order Service      -> Order DB
Payment Service    -> Payment DB
Inventory Service  -> Inventory DB
```

Business transaction:

1. Create order
2. Charge payment
3. Reserve inventory

Each step is its own local transaction.

14. Saga Success Scenario

```
graph TD
    A[Create Order] --> B[SUCCESS]
    B --> C[Charge Payment]
    C --> D[SUCCESS]
    D --> E[Reserve Inventory]
    E --> F[SUCCESS]
```

Final state:

```
Order    -> CREATED
Payment  -> SUCCESS
Inventory -> RESERVED
```

15. Saga Failure Scenario

Suppose the first two operations succeed but inventory reservation fails.

```
Create Order    -> SUCCESS
Charge Payment  -> SUCCESS
Reserve Inventory -> FAILED
```

The earlier database transactions have already committed.

A normal rollback cannot simply undo those commits.

Instead, Saga executes **compensating transactions**.

```
graph TD
    A[Reserve Inventory FAILED] --> B[Refund Payment]
    B --> C[Cancel Order]
```

Final business state:

```
Order    -> CANCELLED
Payment  -> REFUNDED
Inventory -> NOT RESERVED
```

16. What Is a Compensating Transaction?

A compensating transaction is not a database rollback of an already committed transaction.

It is a new business transaction that logically reverses a previous action.

Examples:

Original Action	Compensation
-----	-----
Create order	Cancel order
Charge payment	Refund payment
Reserve inventory	Release inventory
Book flight	Cancel booking
Allocate credit	Release credit

This distinction is very important in interviews.

17. Simple Saga Example

A simplified synchronous example could look like:

```
public void createOrder(OrderRequest request) {  
    String orderId = orderService.createOrder(request);  
  
    String paymentId = null;  
  
    try {  
        paymentId = paymentService.charge(  
            orderId,  
            request.amount());  
  
        inventoryService.reserve(  
            orderId,  
            request.items());  
    } catch (Exception ex) {  
        if (paymentId != null) {  
            paymentService.refund(paymentId);  
        }  
  
        orderService.cancelOrder(orderId);  
  
        throw ex;  
    }  
}
```

This illustrates the concept, but a production Saga requires more robust failure handling.

18. Why Simple try/catch Is Not Enough

Consider:

```
Create Order      -> SUCCESS
Charge Payment    -> SUCCESS
Reserve Inventory -> FAILED
Refund Payment    -> FAILED
```

Now compensation itself has failed.

If Saga state exists only in application memory and the application crashes, the system may lose track of what still needs to be compensated.

Therefore production Saga implementations generally require:

- Persistent state
- Retry handling
- Idempotency
- Failure tracking
- Dead-letter/error handling where messaging is used
- Monitoring and observability

19. Retry

Temporary failures should often be retried.

Example:

```
Refund payment
|
Attempt 1 -> Connection timeout
|
Attempt 2 -> Service unavailable
|
Attempt 3 -> SUCCESS
```

A retry policy commonly includes:

- Maximum retry count
- Exponential backoff
- Delay/jitter
- Classification of retryable vs non-retryable exceptions

Example concept:

```
for (int attempt = 1; attempt <= 3; attempt++) {
    try {
        refundPayment(paymentId);
        break;
    } catch (TemporaryException ex) {
        if (attempt == 3) {
            throw ex;
        }
    }
}
```

In production, a retry library/framework or messaging infrastructure is usually preferable to hand-written retry loops.

20. Idempotency

Retry creates another problem: an operation may be executed more than once.

Example:

```
Refund request sent
  |
  v
Payment service refunds successfully
  |
  v
Network timeout occurs before caller receives response
  |
  v
Caller retries refund
```

Without idempotency:

```
First refund -> 1000
Second refund -> 1000

Customer receives 2000 incorrectly.
```

With idempotency:

```
Idempotency Key: REFUND-PAYMENT-1001
```

Payment service stores the processed key.

```
public RefundResponse refund(
    String paymentId,
    String idempotencyKey) {

    Optional<Refund> existing =
        refundRepository.findByIdempotencyKey(idempotencyKey);

    if (existing.isPresent()) {
        return toResponse(existing.get());
    }

    Refund refund = performRefund(paymentId);

    refundRepository.save(
        refund.withIdempotencyKey(idempotencyKey));

    return toResponse(refund);
}
```

Now retrying the same logical request does not create another refund.

21. Persistent Saga State

Saga progress should normally be persisted so the workflow can recover after application restart or infrastructure failure.

Example Saga state:

Saga ID: SAGA-12345

```
CREATE_ORDER      -> SUCCESS
CHARGE_PAYMENT    -> SUCCESS
RESERVE_INVENTORY -> FAILED
REFUND_PAYMENT    -> PENDING
CANCEL_ORDER      -> PENDING
```

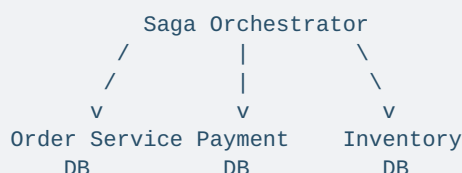
Possible database table:

saga_id	step	status
12345	CREATE_ORDER	SUCCESS
12345	CHARGE_PAYMENT	SUCCESS
12345	RESERVE_INVENTORY	FAILED
12345	REFUND_PAYMENT	PENDING
12345	CANCEL_ORDER	PENDING

After restart, the system can identify unfinished Saga instances and continue processing.

22. Saga Orchestration

In Saga orchestration, one component controls the workflow.



Example flow:

```
Orchestrator -> Create Order
               <- SUCCESS

Orchestrator -> Charge Payment
               <- SUCCESS

Orchestrator -> Reserve Inventory
               <- FAILURE

Orchestrator -> Refund Payment
               <- SUCCESS

Orchestrator -> Cancel Order
               <- SUCCESS
```

Advantages

- Workflow is explicit.
- Easier to understand complex business flows.
- Easier central monitoring.
- Compensation order can be controlled clearly.

Consideration

The orchestrator becomes an important workflow component and must itself be reliable.

23. Saga Choreography

In choreography, there is no single central orchestrator.

Services react to events.

```
Order Service
  |
  | OrderCreated
  v
Payment Service
  |
  | PaymentCompleted
  v
Inventory Service
```

Failure example:

```
Inventory Service
  |
  | InventoryReservationFailed
  v
Payment Service
  |
  | PaymentRefunded
  v
Order Service
  |
  v
Order Cancelled
```

Messaging platforms such as Kafka are often used for this style.

Advantages

- Loose coupling
- Natural event-driven model
- No central orchestrator

Challenges

- Complex event flows can become difficult to understand.
- Debugging distributed workflows can be harder.
- Good observability and correlation IDs are essential.

24. Transactional Outbox Pattern

A common challenge occurs when a service needs to update its database and publish an event.

Incorrect approach:

1. Insert Order into DB -> SUCCESS
2. Publish OrderCreated event -> FAILURE

Now the database says the order exists but downstream services never receive the event.

The reverse order also has a problem:

1. Publish event -> SUCCESS
2. DB commit -> FAILURE

Downstream services receive an event for data that does not exist.

The **Transactional Outbox Pattern** addresses this problem.

In one local database transaction:

```
BEGIN TRANSACTION

Insert Order
Insert Outbox Event

COMMIT
```

Example tables:

```
ORDER
-----
ORDER_ID | STATUS
1001     | CREATED

OUTBOX
-----
EVENT_ID | TYPE           | STATUS
5001     | ORDER_CREATED | NEW
```

A separate publisher reads the outbox table and publishes the event.

After successful publishing:

```
OUTBOX status -> PUBLISHED
```

This is frequently combined with Saga architectures.

25. Saga vs XA/JTA

Area	Saga	XA/JTA / 2PC
Transaction type	Multiple local transactions	One coordinated distributed transaction
Failure recovery	Compensation	Rollback
Typical architecture	Microservices	Tightly coordinated systems
Consistency model	Usually eventual consistency	Stronger atomic consistency
Coupling	Lower	Higher
Resource locking	Local/short transactions	Can hold distributed resources longer
Scalability	Generally more suitable for distributed services	More difficult at large distributed scale
Business complexity	Compensation logic required	Infrastructure complexity required
Retry/idempotency	Essential	Less central to atomic DB commit itself
Kafka/event driven usage	Natural fit	Less natural

26. Most Important Difference

Saga

```
DB1 COMMIT
  |
  v
DB2 COMMIT
  |
  v
DB3 FAIL
  |
  v
Compensate DB2
  |
  v
Compensate DB1
```

The transactions have already committed.

The system performs new business operations to reverse their effects.

XA / Two-Phase Commit

```
DB1 PREPARE
DB2 PREPARE
DB3 PREPARE
```

```
    |
    v
Are all ready?
```

```
YES -----> COMMIT ALL
```

```
NO -----> ROLLBACK ALL
```

The coordinator does not proceed with the final distributed commit unless the resources have successfully completed the prepare stage.

27. Which Approach Should I Choose?

One database

Prefer a local transaction.

```
@Transactional
or
TransactionTemplate
or
PlatformTransactionManager
```

If Spring transaction annotations cannot be used, `TransactionTemplate` is normally a good option.

Two XA-capable databases with strict atomicity requirement

XA/JTA may be considered.

```
Application
|
Transaction Manager
|
+---- XA DB1
|
+---- XA DB2
```

Use it when strong cross-resource atomicity is more important than the additional infrastructure cost and coupling.

Independent microservices

Prefer Saga in most cases.

```
Order Service
|
Payment Service
|
Inventory Service
```

Each service owns its transaction and database.

Use:

- Local transactions
- Saga
- Compensation
- Retry
- Idempotency
- Persistent Saga state
- Transactional outbox where appropriate

28. What If DB1 Commits and DB2 Fails?

This is a very common senior interview follow-up.

A strong response is:

If DB1 and DB2 are independent resources, a normal local transaction cannot automatically roll back DB1 after DB1 has already committed. The solution depends on the architecture. If strict atomicity is required and both resources support distributed transactions, XA/JTA with two-phase commit can coordinate them. In a microservice architecture, I would normally use a Saga. If DB1 succeeds and DB2 fails, I execute a compensating transaction for the DB1 operation. I also make the operation idempotent, retry temporary failures, and persist Saga state so recovery can continue after application restarts.

29. What If the Compensating Transaction Fails?

Example:

```
Create Order      -> SUCCESS
Charge Payment    -> SUCCESS
Reserve Inventory -> FAILED
Refund Payment    -> FAILED
```

Do not simply assume compensation always succeeds.

A robust system should:

1. Persist the Saga state.
2. Mark the compensation step as failed/pending.
3. Retry temporary failures.
4. Use exponential backoff.
5. Ensure the operation is idempotent.

6. Send unrecoverable operations to an error/dead-letter flow if using messaging.
7. Alert or expose the failure through monitoring.
8. Support manual intervention for unrecoverable business failures if necessary.

Example state:

```
SAGA-12345

CREATE_ORDER      SUCCESS
CHARGE_PAYMENT    SUCCESS
RESERVE_INVENTORY FAILED
REFUND_PAYMENT    RETRYING
CANCEL_ORDER      WAITING
```

30. What If the Application Crashes During the Saga?

If the state only exists in memory, the process may be lost.

This is why the state should be persisted.

Example:

Application crashes after payment

Before crash:

```
CREATE_ORDER      SUCCESS
CHARGE_PAYMENT    SUCCESS
INVENTORY         NOT_STARTED
```

After restart:

Read incomplete Saga from DB

|
v

Continue inventory reservation

OR

Trigger compensation according to the workflow rules

Persistent state makes the workflow recoverable.

31. What If an Event Is Received Twice?

Distributed systems usually assume duplicate delivery is possible.

Example:

```
PaymentCompleted event
PaymentCompleted event
```

The consumer should make processing idempotent.

Example:

```
public void handlePaymentCompleted(PaymentCompletedEvent event) {
    if (processedEventRepository.existsById(event.eventId())) {
        return;
    }

    processEvent(event);

    processedEventRepository.save(
        new ProcessedEvent(event.eventId()));
}
```

This avoids processing the same event twice.

32. Common Interview Trap: `@Transactional` Across Microservices

Do not say:

I will add `@Transactional` to Service A and it will roll back Service B.

This is incorrect when Service A and Service B have independent databases and transactions.

Example:

```
Order Service
@Transactional
  |
  | REST call
  v
Payment Service
  |
  v
Payment DB
```

The transaction opened by Order Service does not automatically become the database transaction used inside Payment Service.

Each service normally owns its own local transaction.

33. Common Interview Trap: Catching an Exception and Hiding It

Consider:

```
public void createOrder() {
    try {
        saveOrder();
        processPayment();
    } catch (Exception ex) {
        log.error("Error", ex);
    }
}
```

If transaction management depends on an exception propagating and the exception is swallowed, the transaction may not behave as expected.

If an error should cause rollback, either:

- propagate the exception, or
- explicitly mark the transaction rollback-only when using programmatic transaction management.

Example:

```
transactionTemplate.executeWithoutResult(status -> {
    try {
        saveOrder();
        processPayment();
    } catch (Exception ex) {
        status.setRollbackOnly();
        throw ex;
    }
});
```

34. Transaction Rollback and Checked Exceptions

A useful interview follow-up is the behavior of declarative Spring transactions.

By default, Spring's standard declarative rollback behavior is typically associated with unchecked exceptions (`RuntimeException`) and `Error`.

For checked exceptions, rollback behavior may need explicit configuration when using annotations.

Example:

```
@Transactional(rollbackFor = Exception.class)
public void process() throws Exception {
    // ...
}
```

With programmatic transaction management, you explicitly control rollback behavior.

35. Transaction Propagation - Senior Follow-Up

Interviewers may ask what happens when one transactional method calls another.

Common propagation modes include:

REQUIRED
REQUIRES_NEW
SUPPORTS
MANDATORY
NOT_SUPPORTED
NEVER
NESTED

The most important two to explain are often:

REQUIRED

Existing transaction?

YES -> Join it
NO -> Create new transaction

REQUIRES_NEW

Existing transaction?

YES -> Suspend it
 Create new transaction
NO -> Create new transaction

Example use case:

```
Main business transaction
|
+---- audit operation using REQUIRES_NEW
```

The audit transaction can commit independently from the main transaction, depending on the desired business semantics.

36. Transaction Isolation - Senior Follow-Up

Rollback is only one transaction concern.

Interviewers may also ask about isolation levels.

Typical concepts include:

Dirty Read
Non-repeatable Read
Phantom Read

Common isolation levels:

READ_UNCOMMITTED
READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE

The database and use case determine the appropriate isolation level.

Higher isolation can improve consistency but usually reduces concurrency.

37. Recommended Interview Answer - 30 Seconds

If `@Transactional` is not available, I can manage the transaction programmatically. In Spring, I would normally use `TransactionTemplate` or `PlatformTransactionManager`; with plain JDBC I can disable auto-commit and explicitly call `commit()` or `rollback()`. If all operations are against one database, that gives us a normal local transaction. If the business operation spans multiple independent databases or microservices, a normal local rollback is not sufficient. For XA-capable resources requiring strict atomicity, JTA/XA with two-phase commit is one option. In microservices, I would normally use a Saga with compensating transactions, retry, idempotency, and persisted workflow state.

38. Recommended Interview Answer - 1 to 2 Minutes

In Java, rollback does not depend only on the `@Transactional` annotation. `@Transactional` is Spring's declarative way of managing transaction boundaries. If it cannot be used, I can manage the transaction programmatically. With JDBC I can call `setAutoCommit(false)`, execute all database operations using the same connection, commit when everything succeeds, and call rollback when an exception occurs. In a Spring application, I would generally prefer `TransactionTemplate` or `PlatformTransactionManager` because they integrate with Spring's transaction infrastructure.

If multiple databases are involved, I first identify whether we need one atomic transaction or whether they belong to independent services. If two XA-capable resources require strict atomicity, JTA/XA and two-phase commit can coordinate them. The transaction manager performs a prepare phase and commits only when all participants are ready.

In microservices, however, I would normally avoid keeping a distributed database transaction across services. Instead I would use a Saga. Each service commits its own local transaction, and if a later step fails, previously completed steps are reversed through compensating business transactions. For example, if an order is created and payment succeeds but inventory reservation fails, I refund the payment and cancel the order. The Saga should be persistent, retryable, and idempotent so it can recover from duplicate messages, temporary failures, or application restarts.

39. Interview Example - E-Commerce Order

Suppose the interviewer gives this problem:

User places an order.

1. Save Order
2. Charge Payment
3. Reserve Inventory
4. Generate Shipment

A senior-level design could be:

```
Order Service
|
v
Order DB
|
| OrderCreated
v
Payment Service
|
v
Payment DB
|
| PaymentCompleted
v
Inventory Service
|
v
Inventory DB
|
| InventoryReserved
v
Shipping Service
```

If Inventory fails:

```
InventoryReservationFailed
|
v
Refund Payment
|
v
Cancel Order
```

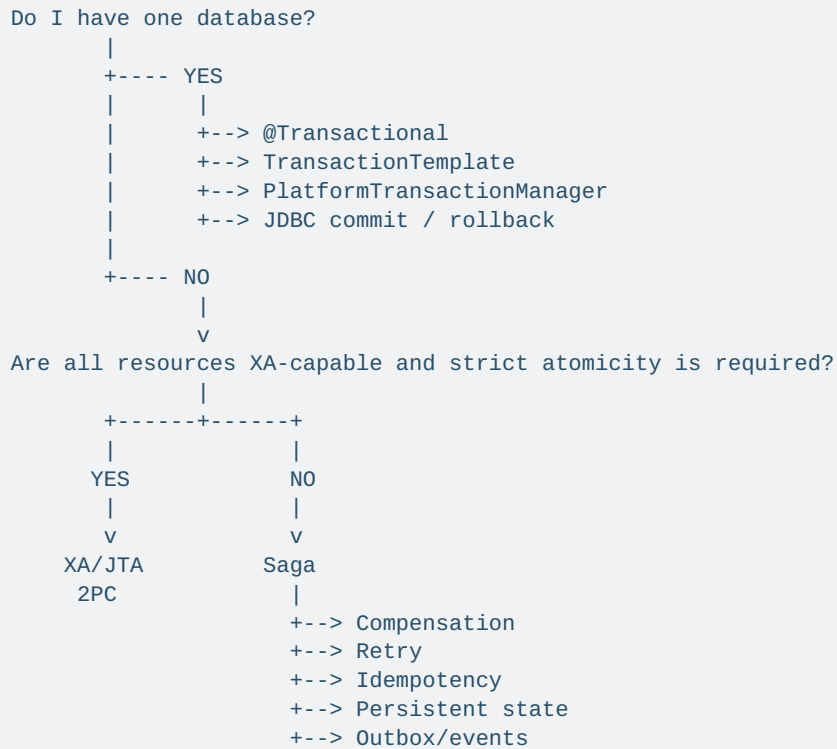
Reliability mechanisms:

- Saga ID / Correlation ID
- Idempotency key
- Persistent Saga state
- Retry with backoff
- Transactional outbox
- Dead-letter handling
- Monitoring and alerts

This demonstrates distributed transaction knowledge beyond simply using annotations.

40. Key Points to Remember

For interviews, remember this decision tree:



41. One-Line Memory Points

@Transactional	= Declarative transaction management
TransactionTemplate	= Programmatic Spring transaction management
JDBC rollback	= Manual local transaction management
XA/JTA	= Distributed atomic transaction coordination
2PC	= Prepare first, then commit all participants
Saga	= Local transactions + business compensation
Compensation	= New transaction that reverses business effect
Idempotency	= Same request can safely execute multiple times
Retry	= Recover from temporary failures
Persistent state	= Resume distributed workflow after crash/restart
Outbox	= Atomically save DB change + event to publish later

42. Final Interview Summary

A 9-10 year Java developer should not answer this question only with:

```
connection.rollback();
```

The interviewer is usually testing whether you understand that rollback strategy changes depending on the transaction boundary.

The complete thought process should be:

Local DB transaction

-> commit / rollback

Spring without @Transactional

-> TransactionTemplate / PlatformTransactionManager

Multiple XA resources requiring atomicity

-> XA/JTA / 2PC

Independent microservices

-> Saga / compensation

+ retry

+ idempotency

+ persistent state

+ observability

That demonstrates both **Java/Spring transaction knowledge** and **distributed-system design understanding**.